

Reviewer Pack — Lorentzian Defect of Tseitin Moment Polynomial Lower-Bounds ...

Entry 2714b4a401ee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 23:41:35 UTC

Lorentzian Defect of Tseitin Moment Polynomial Lower-Bounds SOS Degree

Entry ID: 2714b4a401ee **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 23:41:35 UTC

1. Conjecture

Field A (mathematical branch): Lorentzian polynomials and Hodge-theoretic combinatorics (Brändén-Huh 2020 signature theory) **Field B** (complexity object): Sum-of-Squares (Lasserre) refutation degree of Tseitin Max-CUT on expanders

Statement:

For a d -regular graph $G=(V,E)$ with odd parity $\sigma:V\rightarrow\{0,1\}$ (so the Tseitin formula $T(G,\sigma)$ is unsatisfiable), define the degree-2 Tseitin moment polynomial $P_{\{G,\sigma\}}(x)=\sum_{\{e,f\}\in E} c_{\{e,f\}}(\sigma)\cdot x_e x_f$ with $c_{\{e,f\}}(\sigma)=|\{v\in e\cap f: \sigma(v)=1\}|$ if e,f share a vertex and 0 otherwise. Let $\delta(G,\sigma)=(\text{number of strictly positive eigenvalues of the Hessian } H_{\{P_{\{G,\sigma\}}\}}(1))-1$ be the Lorentzian defect. Conjecture: for every d -regular Ramanujan expander G ($\lambda_2(G)\leq 2\sqrt{d-1}$) and every odd σ , $\delta(G,\sigma) \geq [\alpha\cdot|E|/d^2]$ for an absolute constant $\alpha\geq 1/8$, and the resolution width $w(T(G,\sigma))$ (a known SOS-degree lower bound) satisfies $w(T(G,\sigma)) \geq 2+2\delta(G,\sigma)$; a single (G,σ) violating either inequality refutes the conjecture.

Rationale (proposer's reasoning):

Lorentzian polynomials are precisely those whose Hessian has signature $(1,m-1)$ at all positive points, a rigid algebraic property tied to Hodge-Riemann relations on matroids. Expansion forces the Tseitin parity obstruction to scatter across many edge-pairs, so the contribution matrix should have many positive eigendirections—a defect that should reappear as algebraic hardness in SOS, since SOS reductions preserve the moment-matrix signature up to bounded distortion. This connects Brändén-Huh signature combinatorics, which (to our knowledge) has not been used in proof complexity, to the open Tseitin/SOS lower-bound program.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 284b8e8d4f472d21

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For $d=3$ and $n \in \{8, 12, 16, 20, 24, 28, 32, 36, 40\}$, draw 30 Ramanujan-ish 3-regular graphs per n with random odd σ . Compute Lorentzian defect δ via eigh of the $|E| \times |E|$ Hessian $(2c_{\{e,f\}})$ and resolution width w . Support iff in $\geq 27/30$ trials per n both $\delta \geq \lfloor |E|/(8 \cdot 9) \rfloor$ and $w \geq 2 + 2\delta$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Lorentzian polynomials Tseitin sum-of-squares degree expander - Brändén Huh signature Hessian Tseitin Max-CUT Lasserre lower bound - resolution width Tseitin Ramanujan expander SOS refutation moment polynomial

Top relevant hits considered: - [<http://arxiv.org/abs/2304.08399v3>] Dually Lorentzian Polynomials - [<http://arxiv.org/abs/2206.02759v4>] Polynomials with Lorentzian Signature, and Computing Permanents via Hyperbolic Programming - [<http://arxiv.org/abs/1209.2976v3>] Sums of squares of polynomials with rational coefficients

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_random_odd_sigma(n):
    sigma = [random.choice([0, 1]) for _ in range(n)]
    if sum(sigma) % 2 == 0:
        sigma[random.randint(0, n-1)] = 1 - sigma[-1]
    return sigma

def is_ramanujanish(G, d, lambda_2_threshold):
    degree_sum = sum(degree for _, degree in G.items())
    if degree_sum != len(G) * d:
        return False
    lambda_2 = max(eigenvalue for eigenvalue in get_eigenvalues(G) if eigenvalue > 0)
    return lambda_2 <= lambda_2_threshold

def generate_random_d_regular_graph(n, d):
    G = {}
    edges = set()
```

```

while len(edges) < n * (d // 2):
    u = random.randint(0, n-1)
    v = random.randint(0, n-1)
    if u != v and (u, v) not in edges and (v, u) not in edges:
        G[u] = G.get(u, set()) | {v}
        G[v] = G.get(v, set()) | {u}
        edges.add((u, v))
return G

def get_eigenvalues(G):
    n = len(G)
    A = [[0 for _ in range(n)] for _ in range(n)]
    for u, neighbors in G.items():
        for v in neighbors:
            A[u][v] += 1
            A[v][u] += 1
    return eigenvalues(A)

def eigenvalues(matrix):
    n = len(matrix)
    if n == 0:
        return []
    if n == 1:
        return [matrix[0][0]]

    def determinant(submatrix):
        if len(submatrix) == 2:
            return submatrix[0][0] * submatrix[1][1] - submatrix[0][1] * submatrix[1][0]
        det = 0
        for j in range(len(submatrix)):
            det += (-1) ** j * submatrix[0][j] * determinant([row[:j] + row[j+1:] for row in submatrix[1:]])
        return det

    def characteristic_polynomial(matrix):
        n = len(matrix)
        if n == 2:
            return [matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0], -(matrix[0][0] + matrix[1][1])]
        else:
            char_poly = []
            for j in range(n):
                submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
                char_poly.append((-1) ** (n-1-j) * determinant(submatrix))
            return char_poly

    def roots(poly):
        if len(poly) == 1:
            return []
        elif len(poly) == 2:
            return [-poly[0] / poly[1]]
        else:
            a, b, c = poly[-3], poly[-2], poly[-1]
            discriminant = b**2 - 4*a*c
            if discriminant < 0:
                real_part = -b / (2*a)
                imaginary_part = math.sqrt(-discriminant) / (2*a)
                return [real_part + imaginary_part * 1j, real_part - imaginary_part * 1j]

```

```

        elif discriminant == 0:
            return [-b / (2*a)]
        else:
            sqrt_discriminant = math.sqrt(discriminant)
            return [(-b + sqrt_discriminant) / (2*a), (-b - sqrt_discriminant) / (2*a)]

char_poly = characteristic_polynomial(matrix)
return roots(char_poly)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 12, 16, 20, 24, 28, 32, 36, 40]
    d = 3
    lambda_2_threshold = 2 * math.sqrt(d - 1) + 0.4
    alpha = 1 / 8

    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        ramanujanish_graphs = [generate_random_d_regular_graph(n, d) for _ in range(30)]
        ramanujanish_graphs = [G for G in ramanujanish_graphs if is_ramanujanish(G, d, lambda_2_th

        for G in ramanujanish_graphs:
            sigma = generate_random_odd_sigma(n)
            P_Gsigma = [[0] * n for _ in range(n)]
            for e, f in combinations(range(n), 2):
                if e in G and f in G[e]:
                    c_ef_sigma = sum(1 for v in G[e] & G[f] if sigma[v] == 1)
                    P_Gsigma[e][f] = 2 * c_ef_sigma
            H = [[0] * n for _ in range(n)]
            for e, f in combinations(range(n), 2):
                H[e][f] = P_Gsigma[e][f]
                H[f][e] = P_Gsigma[e][f]

            eigenvalues_H = get_eigenvalues(H)
            delta = sum(1 for eigenvalue in eigenvalues_H if eigenvalue > 0) - 1
            metric_values.append(delta)
            instances_tested += 1

            if delta < n / (8 * d**2):
                conjecture_holds = False
                counterexample = f"n={n}, δ={delta} < {n / (8 * d**2)}"

        w_TGsigma = 0
        for width in range(2, math.ceil(math.log2(n)) + 1):
            if is_refutation(G, sigma, width):
                w_TGsigma = width
                break

        metric_values.append(w_TGsigma)
        instances_tested += 1

        if w_TGsigma < 2 + 2 * delta:

```

```

        conjecture_holds = False
        counterexample = f"n={n}, w={w_TGsigma} < {2 + 2 * delta}"

    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = instances_tested // 30

    return {
        "metric_name": "Lorentzian defect and resolution width",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def is_refutation(G, sigma, width):
    stack = [(0, 0)]
    visited = set()

    while stack:
        node, level = stack.pop()
        if level >= width or node in visited:
            continue
        visited.add(node)

        for neighbor in G[node]:
            if neighbor not in visited:
                stack.append((neighbor, level + 1))

    return len(visited) == len(G)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_543310d6.py", line 181, in <module>
    result = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_543310d6.py", line 112, in run_trial
    ramanujanish_graphs = [G for G in ramanujanish_graphs if is_ramanujanish(G, d, lambda_2_threshold)]
                        ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_543310d6.py", line 24, in is_ramanujanish
    degree_sum = sum(degree for _, degree in G.items())

```

